Raport

Wybór najlepszego modelu klasyfikacyjnego

Adrian Krzyżanowski

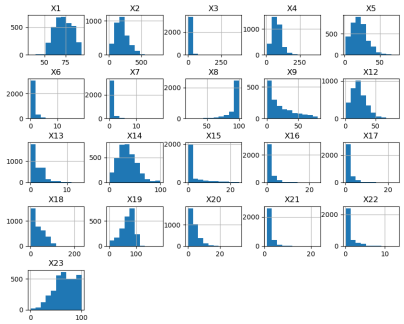
23 grudnia 2024

0.1 Wstęp

Celem projektu jest przygotowanie modelu klasyfikacyjnego o jak największej mocy predykcyjnej, kierując się miara zrównoważonej dokładności (balanced accuracy). Model został przygotowany na podstawie danych dotyczących przewidywania szans na niewypłacalność.

0.2 Przygotowanie danych

Nasz zbiór treningowy (część X) składa się z 3660 wierszy i 23 kolumn, z czego dwie zmienne są kategoryczne (X10 i X11), a pozostałe są numeryczne. Zbiór posiada braki danych (w postaci ujemnych wartości), które stanowią około 11% wszystkich wartości. Ponieważ różne modele różnie radzą sobie z brakami danych, więc odpowiednie uzupełnianie braków danych (imputer) zostanie wybrane przy projektowaniu poszczególnych modeli. Jeśli chodzi o rozkłady zmiennych to wyglądają one następująco:



Rysunek 1: Histogramy

Widzimy, że rozkłady są skośne i gdzieniegdzie pojawiają się outliery, więc dane muszą zostać przeskalowane, tym również zajmiemy się przy testowaniu modeli.
Natomiast jeśli chodzi o zbiór testowy to tutaj braki danych stanowią około 5% całego zbioru (1285 wierszy).
W etykietach zbioru treningowego (część y) mamy dwie klasy: "Good" oraz "Bad", więc pierwszym krokiem przetwarzania danych będzie zamiana tych klas na zmienne typu bool, gdzie True oznacza "Bad", a False - "Good". Następnie w zbiorze danych (części X) zamieniamy wartości ujemne (braki danych) na NaN oraz zmieniamy typ zmiennych numerycznych dla X10 i X11 na kategoryczny. Jak wiemy z opisu niektóre kategorie w zmiennej X10 się pokrywają, więc przyporządkowujemy je do wspólnych kategorii, zmniejszając ich ogólną liczbę.

Poza tym w etykietach zbioru treningowego nie ma dysproporcji klas (różnica to około 5% ogólnej liczby).

0.3 Część eksperymentalna

0.3.1 Wstęp

Głównym podejściem do wyznaczenia najlepszego modelu było po prostu rozpatrzenie wszystkich rodzajów dotychczas poznanych modeli, dla każdego rodzaju przygotowanie jak najlepszego modelu w oparciu o dane i porównanie ich ze sobą względem miary balanced accuracy.

0.3.2 Realizacja

Wśród rozpatrywanych rodzajów modeli były:

1. Drzewa decyzyjne	2. Regresja logistyczna	3. SVM
4. LDA	5. QDA	6. Naiwny Bayes
7. K najbliższych sąsiadów	8. Las losowy	9. Bagging
10. ExtraTrees	11. Gradient boosting	12. VotingClassifier

Na początku zmienne zostały rozróżnione na numeryczne i kategoryczne. Dla każdego rodzaju modelu należało dopasować jak najlepsze skalowanie dla zmiennych numerycznych oraz uzupełnianie brakujących danych (imputer) dla obu typów zmiennych. Dodatkowo zmienne kategoryczne zostały zakodowane za pomocą OneHotEncoding. Wśród rozważanych sposobów skalowania były:

StandardScaler	MinMaxScaler	MaxAbsScaler
RobustScaler	Normalizer	QuantileTransformer
PowerTransformer		

Natomiast wśród rozważanych imputerów były: SimpleImputer, KNNImputer. Dla SimpleImputer rozważony został parametr strategy - 'mean', 'most_frequent', 'constant', 'median', a dla KNNImputer - n_neighbors i weights o wartościach odpowiednio: [3, 5, 7, 9, 11, 13, 15, 20, 30] i ["uniform", 'distance']. Parametry dla poszczególnych modeli przedstawiają się następująco:

1. Drzewo decyzyjne: criterion, max_depth, min_samples_leaf
2. Regresja logistyczna: penalty, tol, class_weight, max_iter, C, solver
3. SVM: kernel, C, tol, class_weight, max_iter
4. LDA: solver, tol

5. QDA: tol
6. Naive Bayes: alpha
7. KNN: n_neighbors, weights, algorithm, leaf_size, p
8. RandomForest: n_estimators, criterion, max_depth, max_features, min_samples_split, min_samples_leaf
9. BaggingClassifier: estimator (modele 1, 2, 8, 10, 11, 7), n_estimators, max_samples, bootstrap
10. ExtraTrees: n_estimators, max_depth, min_samples_split, min_samples_leaf, bootstrap, criterion
11. GradientBoosting: n_estimators, learning_rate, max_depth, min_samples_split, min_samples_leaf, subsample, max_feature, loss, criterion
12. VotingClassifier: estimators (modele 2, 8, 7, 11), voting

Dla każdego rodzaju modelu rozpatrzono heurystycznie wszystkie możliwe kombinacje za pomocą Gridsearch z kroswalidacją (3 do 7 krotną, w zależności od długości wykonywania się obliczeń) i parametrem scoring ustawionym na balanced_accuracy. Heurystycznie, gdyż obliczenia wykonywały się zbyt długo. Najpierw została wybrana metoda skalowania, przy czym wszystko inne było ustawione na domyślne. Potem imputer, dalej najlepsze parametry dla imputera, a na koniec najlepsze parametry dla modelu, które były rozpatrywane częściowo w przypadku ich dużej liczby.

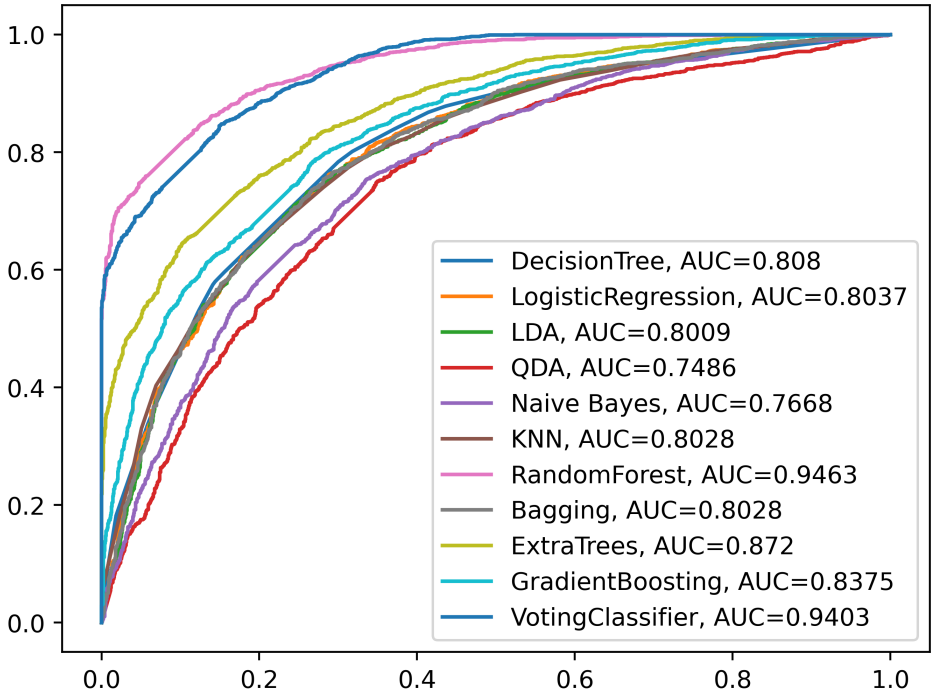
0.3.3 Wyniki

Po przygotowaniu najlepszego modelu każdego rodzaju zostały policzone predykcje na zbiorze treningowym, a następnie obliczono balanced accuracy. Wyniki przedstawiają się następująco:

	Model	balanced_accuracy_score
1	DecisionTree	0.741741
2	LogisticRegression	0.736307
3	SVM	0.572308
4	LDA	0.730833
5	QDA	0.645483
6	Naive Bayes	0.705632
7	KNN	0.729509
8	RandomForest	0.861788
9	BaggingClassifier	0.729956
10	ExtraTrees	0.783396
11	GradientBoosting	0.753535
12	VotingClassifier	0.83957

Rysunek 2: Tabela

Następnie policzono krzywe ROC i miary AUC dla każdego z modeli (oprócz SVM), co można zaobserwować na poniższym wykresie.



Rysunek 3: Krzywe ROC

0.3.4 Wnioski

Z tabeli oraz krzywych ROC wynika, że dla podanych danych najlepszy jest model RandomForest. Ma najwyższy wynik balanced_accuracy oraz roc_auc. Poza tym nie jest to za wysoki wynik dla danych treningowych (poniżej 0.95), więc model raczej się nie przeuczył. Można by było jeszcze rozpatrzeć więcej wartości dla parametrów, jednak nie wpłynęłoby to znacznie na wynik. Co ciekawe gdy policzymy balanced_accuracy dla modelu domyślnego RandomForest (czyli takiego bez dopasowanych parametrów do danych) to otrzymamy wynik 0.9760945, co oznacza, że model najpewniej się przeuczył i nie osiągnie takiego rezultatu na danych testowych.